

DA6401W - Assignment 4

Department of Data Science and AI, IIT Madras

Mar 2026

Topics covered: Steepest descent, ADMM, BFGS, L-BFGS, Nesterov, regularization of DNN, data augmentation, label smoothing, noise injection, bagging, dropout, skip connection, early stopping, curriculum training, knowledge distillation and teacher training

Submit your solutions in PDF file with written answers, and jupyter notebook with codes, plots and outputs printed.

1. (Theoretical Question) Consider training a deep neural network using stochastic gradient descent (SGD) on a non-convex loss surface. Curriculum learning is implemented by first training on “easy” examples and gradually introducing harder examples.

Which of the following statements is **most theoretically accurate**?

- (A) Curriculum learning guarantees convergence to a flatter minimum.
- (B) Curriculum learning modifies early gradient statistics, thereby influencing the optimization trajectory in parameter space.
- (C) Curriculum learning changes the global minimum of the objective function.
- (D) Curriculum learning eliminates the effect of random initialization.

Solution: Correct Answer: (C)

Curriculum learning does not change the loss function, architecture, or optimization algorithm. Therefore, it does not alter the global minimum of the objective function, nor does it remove non-convexity.

However, SGD is path-dependent: early gradient updates influence which region (or basin of attraction) in parameter space the optimization trajectory enters.

By presenting easier examples first, curriculum learning changes the distribution and magnitude of early gradients. This modifies the optimization trajectory, even though the objective function remains unchanged.

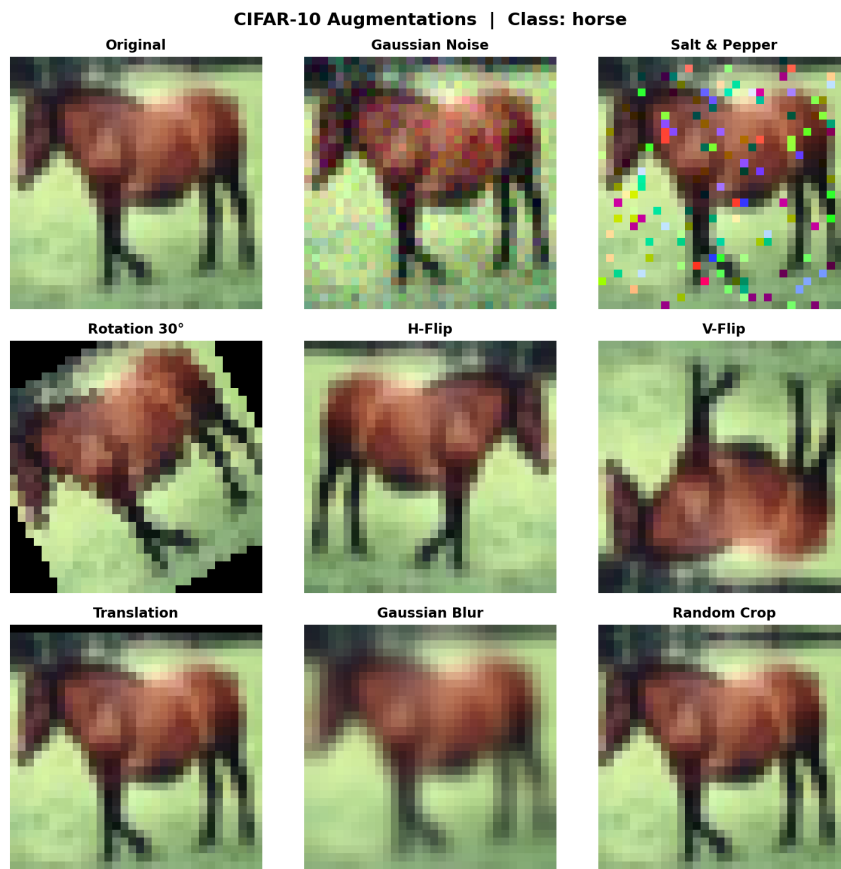
Hence, the most theoretically accurate statement is (C).

2. (Programming Question) Load the CIFAR-10 dataset using `torchvision.datasets.CIFAR10`. Apply the following augmentation transforms individually using `transforms.Compose`, combine all transformed datasets into a single `DataLoader`, and visualize one image under all transformations in a single grid plot. The transformations to apply are:

- (i) Gaussian Noise: add $\epsilon \sim \mathcal{N}(0, 0.05^2)$ to each pixel and clip to $[0, 1]$
- (ii) Salt & Pepper Noise: randomly set pixels to 0 or 1 with probability $p = 0.05$
- (iii) Rotation: rotate by $\theta = 30^\circ$ using `RandomRotation`
- (iv) Horizontal Flip: using `RandomHorizontalFlip(p=1.0)`
- (v) Vertical Flip: using `RandomVerticalFlip(p=1.0)`
- (vi) Translation: shift by `translate=(0.2, 0.15)` using `RandomAffine`
- (vii) Gaussian Blur: `GaussianBlur(kernel_size=3, sigma=1.5)`
- (viii) Random Crop: pad by 4 pixels (reflect mode) then `RandomCrop(32)`

Pick a fixed image index, apply all 8 transforms to that image, and display the original alongside all 8 augmented versions in a 3×3 matplotlib grid.

Solution:



3. (Programming Question) **Bagging Classifier via Maximum Voting on Fashion-MNIST using Pytorch**

Objective: Implement a complete bootstrap aggregation (bagging) training and inference pipeline to observe how ensembling improves generalization over a single model.

Problem Statement: You will train an ensemble of $M = 3$ independent Feed-Forward Neural Networks on the Fashion-MNIST dataset. To simulate bagging, each network must be trained on a unique dataset created by bootstrapping the original training data. Finally, you will evaluate the ensemble on a standard, untouched test set using a maximum voting scheme.

Specific Requirements:

- Load the standard Fashion-MNIST dataset (Pytorch: `torchvision.datasets.FashionMNIST`). 60,000 training (80%) + validation images (20%), 10,000 test images.
- Create three bootstrap version of training data by randomly selecting subsets of images from the training split $T_1, T_2, \& T_3$. Each training sets should have randomly chosen 38400 image, label pairs.
- Train three DNN ($N_1, N_2, \& N_3$) using the three training sets $T_1, T_2, \& T_3$

Model Architecture:

- Feed-Forward Neural Network.
- Input: Flatten the 28x28 images into a 784-dimensional vector.
- Hidden Layer 1: Dense layer with 128 neurons and ReLU activation.
- Hidden Layer 2: Dense layer with 64 neurons and ReLU activation.
- Output Layer: Dense layer with 10 neurons (for the 10 class labels).
- Optimizer: Adam with a learning rate of 0.001.
- Loss Function: Cross-Entropy Loss.
- Batch Size: 64.
- Epochs: 5 per model.

Report the accuracy of the individual models and accuracy of the maximum voting inference ensemble (the class chosen by the majority of the 3 networks wins. If voting tied, randomly select one of the label). Summarize your observation.

Solution: -

4. (Programming Question) **Combating Overfitting with L_2 Regularization (Weight Decay)**

Objective: Investigate model memorization on constrained datasets and evaluate the efficacy of parameter constraints in improving generalization.

Tasks:

1. **Dataset:** Use `torchvision.datasets.FashionMNIST` and flatten it to get a vector of size 784
2. **Model Setup:** Implement an intentionally over-parameterized Multi-Layer Perceptron (MLP) with three hidden layers (e.g., 1024, 512, and 256 units).
3. **Baseline Training (Observation):** Train this network on a random 10% subset of the Fashion MNIST training dataset for 50 epochs. Plot both the training and validation loss curves.
4. **Intervention:** Modify your training loop or optimizer to include L_2 regularization (weight decay). *Hint: If you are using PyTorch, you can apply this automatically by passing the `weight_decay` argument to your optimizer.*
For example: `optim.Adam(model.parameters(), lr=1e-3, weight_decay=lambda_l2)`.
5. **Analysis:** Re-train the model using two new regularization coefficients: $\lambda = 10^{-4}$ and $\lambda = 10^{-2}$. Plot the validation loss curves for all three configurations ($\lambda = 0$, $\lambda = 10^{-4}$, $\lambda = 10^{-2}$) on a single comparative graph. Briefly discuss how the magnitude of λ impacts the model's generalization capabilities.

Solution: -

5. (Conceptual MCQ) Nesterov Accelerated Gradient

Momentum methods are commonly used to accelerate gradient-based optimization in deep learning. Nesterov Accelerated Gradient (NAG) modifies the classical momentum method.

Which of the following statements best describes the key idea behind **Nesterov momentum**?

- (A) The gradient is computed at the current parameter location without using momentum.
- (B) The gradient is computed after performing a *look-ahead* step using the momentum term.
- (C) The learning rate is automatically adjusted based on the magnitude of gradients.
- (D) The optimization step ignores past gradients completely.
- (E) The algorithm guarantees convergence to the global optimum for non-convex problems.

Solution:

Correct Answer: (B)

In classical momentum, the gradient is evaluated at the current parameter location and then combined with the velocity term.

In Nesterov Accelerated Gradient (NAG), the optimizer first performs a *look-ahead step* using the momentum velocity and then computes the gradient at this anticipated future position.

This allows the optimizer to make a corrective adjustment before updating the parameters, often resulting in faster convergence and more stable optimization.

6. (Programming Question) Knowledge Distillation

Knowledge distillation is a technique where a large *teacher model* transfers knowledge to a smaller *student model*.

1. Load the MNIST dataset using `torchvision.datasets.MNIST`.
2. Train a **teacher network** with the following architecture:

$$784 \rightarrow 256 \rightarrow 128 \rightarrow 10$$

Use ReLU activation for hidden layers and Softmax for last layer and train for 5 epochs.

3. Define a smaller **student network**

$$784 \rightarrow 64 \rightarrow 10$$

4. Train the student model using the following distillation loss:

$$L = \alpha L_{\text{CE}} + (1 - \alpha) L_{\text{KD}}$$

where

$$L_{\text{KD}} = KL(p_T^\tau || p_S^\tau)$$

and τ is the temperature.

Use

$$\alpha = 0.5, \quad \tau = 3$$

5. Compare the test accuracy of:
 - Student model trained without a teacher model
 - Student model trained using distillation

Briefly comment on the results.

Solution:

Teacher Model Training

```

teacher = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784,256),
    nn.ReLU(),
    nn.Linear(256,128),
    nn.ReLU(),
    nn.Linear(128,10)
)

```

Student Model

```

student = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784,64),
    nn.ReLU(),
    nn.Linear(64,10)
)

```

Distillation Loss

```

import torch.nn.functional as F

def distillation_loss(student_logits, teacher_logits, labels,
                      alpha=0.5, T=3):

    ce_loss = F.cross_entropy(student_logits, labels)

    kd_loss = F.kl_div(
        F.log_softmax(student_logits/T, dim=1),
        F.softmax(teacher_logits/T, dim=1),
        reduction='batchmean'
    ) * (T*T)

    return alpha*ce_loss + (1-alpha)*kd_loss

```

Observation

The student trained with knowledge distillation typically achieves better generalization than the student trained only with hard labels, because the teacher provides additional information through soft probability distributions.

7. Consider a fully connected neural network trained for a classification task. During training, **dropout** is applied to a hidden layer with dropout probability p .

1. Explain how dropout helps prevent overfitting in neural networks. In your answer, discuss why dropout can be interpreted as implicitly training an ensemble of subnetworks.
2. During training, neurons are randomly dropped with probability p . However, during inference (testing), dropout is not applied.
 - (a) Explain how the network output is adjusted during inference to account for dropout used during training.
 - (b) Why is this scaling necessary?
3. Suppose the output of a neuron before dropout is h . Let the dropout mask be $m \sim \text{Bernoulli}(1 - p)$. The output after dropout during training is

$$\tilde{h} = mh$$

- (a) Compute $E[\tilde{h}]$.
4. Dropout is often applied to fully connected layers but less frequently to convolutional layers in modern architectures. Provide two reasons why dropout may be less effective in convolutional layers.

Solution:

1. Dropout as a Regularization Technique

Dropout prevents overfitting by randomly removing (dropping) neurons during training. At each training iteration, each neuron is kept with probability $1 - p$ and dropped with probability p . As a result, the network cannot rely on specific neurons and must distribute representations across multiple neurons.

This process can be interpreted as implicitly training a large number of different subnetworks. Each dropout mask corresponds to a different subnetwork formed by removing some neurons. During training, different subnetworks are sampled and trained, but they share parameters. At inference time, the full network is used, which approximates averaging the predictions of all these subnetworks. This ensemble-like behavior improves generalization and reduces overfitting.

2. Training vs Inference

(a) Adjustment during inference

During inference, dropout is not applied and all neurons are active. To account for the missing activations during training, the outputs are scaled by the retention probability $1 - p$.

If h is the activation during inference, it becomes

$$h_{\text{test}} = (1 - p)h$$

Alternatively, many implementations use *inverted dropout*, where scaling is performed during training instead of inference.

(b) Why scaling is necessary

Without scaling, the expected magnitude of activations during training would be smaller than during inference. This mismatch would cause the network to produce larger outputs at test time compared to training time. Scaling ensures that the expected activation values remain consistent between training and inference.

3. Expectation Analysis

Given

$$\tilde{h} = mh$$

where $m \sim \text{Bernoulli}(1 - p)$.

(a) Expected value

Since m takes value 1 with probability $1 - p$ and 0 with probability p ,

$$E[\tilde{h}] = E[mh]$$

$$= hE[m]$$

$$= h(1 - p)$$

Thus,

$$E[\tilde{h}] = (1 - p)h$$

4. Dropout in Convolutional Layers

Dropout is less commonly used in convolutional layers for the following reasons:

- **Spatial correlation:** Features in convolutional layers are spatially correlated. Randomly dropping individual activations may not effectively regularize the network because nearby pixels contain similar information.
- **Parameter sharing:** Convolutional layers already have fewer parameters due to weight sharing, which naturally reduces overfitting compared to fully connected layers. Therefore, the regularization benefit of dropout is smaller.

8. **Programming:** In deep neural networks, very deep architectures often suffer from the *vanishing gradient problem*, which makes training difficult. One of the most successful solutions to this issue is the use of **skip connections** (also known as residual connections), introduced

in Residual Networks (ResNets).

In this problem, you will implement a simple neural network with and without a skip connection and compare their training behavior.

1. Generate a synthetic regression dataset where the input $x \in \mathbb{R}^{20}$ and the target is

$$y = \sin(w^T x) + \epsilon$$

where w is a randomly generated vector and $\epsilon \sim \mathcal{N}(0, 0.1)$.

2. Implement a **plain deep neural network** with the following structure:

$$20 \rightarrow 64 \rightarrow 64 \rightarrow 64 \rightarrow 1$$

using ReLU activations.

3. Implement a **residual neural network** with a skip connection between the first and third hidden layers:

$$\begin{aligned}h_1 &= \text{ReLU}(W_1 x) \\h_2 &= \text{ReLU}(W_2 h_1) \\h_3 &= \text{ReLU}(W_3 h_2 + h_1) \\y &= W_4 h_3\end{aligned}$$

The activation from the first hidden layer is added to the output of the third hidden layer, forming the residual connection.

4. Train both networks using the same optimizer(Adam), learning rate(1e-3), and number of epochs(25).
5. Plot the following:
 - Training loss vs epochs for both networks
 - Gradient norm of the first layer during training
6. Briefly comment on the differences observed in training behavior.

Solution:

Step 1: Dataset Generation

```
import torch
import numpy as np

N = 5000
d = 20
```

```

X = torch.randn(N, d)
w = torch.randn(d)

y = torch.sin(X @ w) + 0.1 * torch.randn(N)
y = y.unsqueeze(1)

```

Step 2: Plain Deep Network

```

import torch.nn as nn

class PlainNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(20,64),
            nn.ReLU(),
            nn.Linear(64,64),
            nn.ReLU(),
            nn.Linear(64,64),
            nn.ReLU(),
            nn.Linear(64,1)
        )

    def forward(self,x):
        return self.net(x)

```

Step 3: Residual Network

```

class ResidualBlock(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(64,64)
        self.fc2 = nn.Linear(64,64)
        self.relu = nn.ReLU()

    def forward(self,x):
        out = self.relu(self.fc1(x))
        out = self.fc2(out)
        return self.relu(out) + x

class ResNet(nn.Module):
    def __init__(self):
        super().__init__()

```

```

        self.input = nn.Linear(20,64)
        self.block = ResidualBlock()
        self.output = nn.Linear(64,1)
        self.relu = nn.ReLU()

    def forward(self,x):
        h = self.relu(self.input(x))
        h = self.block(h)
        return self.output(h)

```

Step 4: Training

```

model = PlainNet()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.MSELoss()

for epoch in range(100):
    pred = model(X)
    loss = criterion(pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

```

The same training loop can be used for the residual network.

Step 5: Expected Observation

- The residual network typically converges faster.
- Gradient norms in earlier layers remain larger in the residual network.
- Skip connections allow gradients to flow directly through identity paths, mitigating the vanishing gradient problem.

9. (Programming Question) Early Stopping

Train a simple neural network on the MNIST dataset and implement early stopping.

1. Load MNIST using `torchvision.datasets.MNIST`.
2. Train a small MLP:
 - **Input Layer (784 neurons):** Each MNIST image has size 28×28 . When flattened into a vector, the total number of input features becomes

$$28 \times 28 = 784$$

Thus the input layer contains 784 neurons.

- **Hidden Layer 1 (128 neurons):** The first fully connected layer transforms the 784-dimensional input vector into a 128-dimensional feature representation. A ReLU activation function is applied after this layer.
 - **Hidden Layer 2 (64 neurons):** The second fully connected layer further reduces the representation from 128 features to 64 features, again followed by a ReLU activation.
 - **Output Layer (10 neurons):** The final layer contains 10 neurons corresponding to the 10 digit classes in MNIST (0–9). This layer outputs logits which are used with the cross-entropy loss function during training.
3. Monitor validation loss after every epoch.
 4. Implement early stopping with patience = 5 epochs.
 5. Plot training and validation loss curves.

Solution:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split

transform = transforms.ToTensor()

dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)

train_set, val_set = random_split(dataset, [50000, 10000])

train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
val_loader = DataLoader(val_set, batch_size=64)

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 10)
```

```

)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(),lr=1e-3)

best_loss = float('inf')
patience = 5
counter = 0

for epoch in range(50):

    model.train()
    for x,y in train_loader:
        pred = model(x)
        loss = criterion(pred,y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    model.eval()
    val_loss = 0

    with torch.no_grad():
        for x,y in val_loader:
            val_loss += criterion(model(x),y).item()

    val_loss /= len(val_loader)

    if val_loss < best_loss:
        best_loss = val_loss
        counter = 0
    else:
        counter += 1

    if counter >= patience:
        print("Early stopping triggered")
        break

```

10. (Numerical Question) Label Smoothing

A classification task has $K = 4$ classes. Label smoothing with parameter

$$\epsilon = 0.1$$

is applied.

If the true class is class 2, compute the smoothed target distribution.

Solution:

Correct class probability:

$$1 - \epsilon = 0.9$$

Remaining probability:

$$\epsilon = 0.1$$

Number of other classes:

$$K - 1 = 3$$

Each other class receives:

$$\frac{0.1}{3} \approx 0.0333$$

Final distribution:

$$[0.0333, 0.9, 0.0333, 0.0333]$$

11. Numerical: One Iteration of Steepest Descent

Consider the quadratic objective function

$$f(w_1, w_2) = 2w_1^2 + w_1w_2 + w_2^2$$

The steepest descent update rule is

$$w_{t+1} = w_t - \eta \nabla f(w_t)$$

Given:

$$w_0 = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad \eta = 0.1$$

Compute the updated parameter vector w_1 after one iteration of steepest descent.

Solution:

First compute the gradient.

$$\frac{\partial f}{\partial w_1} = 4w_1 + w_2$$

$$\frac{\partial f}{\partial w_2} = w_1 + 2w_2$$

Thus

$$\nabla f = \begin{bmatrix} 4w_1 + w_2 \\ w_1 + 2w_2 \end{bmatrix}$$

At

$$w_0 = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$$

$$\nabla f(w_0) = \begin{bmatrix} 4(2) + (-1) \\ 2 + 2(-1) \end{bmatrix} = \begin{bmatrix} 7 \\ 0 \end{bmatrix}$$

Update:

$$w_1 = w_0 - \eta \nabla f(w_0)$$

$$= \begin{bmatrix} 2 \\ -1 \end{bmatrix} - 0.1 \begin{bmatrix} 7 \\ 0 \end{bmatrix}$$

$$= \begin{bmatrix} 1.3 \\ -1 \end{bmatrix}$$

$$\boxed{w_1 = [1.3, -1]^T}$$

12. **Programming: Noise Injection as Regularization**

In this problem you will study the effect of noise injection during training.

1. Generate a synthetic regression dataset with

$$x \in \mathbb{R}^{10}, \quad y = w^T x + \epsilon$$

where w is randomly generated and $\epsilon \sim \mathcal{N}(0, 0.1)$.

Use $N = 2000$ samples and split the dataset into 80% training and 20% test data.

2. Implement a neural network with the architecture

$$10 \rightarrow 64 \rightarrow 32 \rightarrow 1$$

using ReLU activations.

3. Train the network using two settings:

- Model A: Standard training
- Model B: Noise injection

For Model B, add Gaussian noise to the inputs during training:

$$\tilde{x} = x + \eta$$

where

$$\eta \sim \mathcal{N}(0, 0.05^2)$$

4. Train both models using

- Optimizer: Adam
- Learning rate: 10^{-3}
- Epochs: 50
- Batch size: 64

5. Plot:

- Training loss vs epochs
- Test loss comparison

6. Briefly discuss how noise injection affects model generalization.

Solution:

Noise injection acts as a regularization method by perturbing the input data during training. This prevents the network from memorizing exact input patterns and encourages the model to learn smoother functions that are robust to small variations in the input.

Typically, the model trained with noise injection shows slightly higher training loss but better test performance, indicating improved generalization.

13. Numerical: One Iteration of ADMM for Lasso Problem

Note (ADMM intuition): The Alternating Direction Method of Multipliers (ADMM) is an optimization algorithm used to solve problems where the objective function can be split into multiple components with simple structure.

In ADMM, the constrained problem

$$\min_{w,z} f(w) + g(z) \quad \text{subject to } w = z$$

is solved by performing *alternating updates*:

- A quadratic minimization step for w
- A proximal (soft-thresholding) step for z
- A dual variable update enforcing consistency between w and z

For the Lasso problem, the z -update corresponds to applying the **soft-thresholding operator**, which promotes sparsity in the solution.

Consider the Lasso optimization problem

$$\min_w \frac{1}{2} \|Aw - b\|_2^2 + \lambda \|z\|_1$$

subject to

$$w = z$$

Using the ADMM formulation, the augmented Lagrangian is

$$\mathcal{L}_\rho(w, z, u) = \frac{1}{2} \|Aw - b\|_2^2 + \lambda \|z\|_1 + \frac{\rho}{2} \|w - z + u\|_2^2$$

where u is the scaled dual variable.

Given:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Initial values:

$$w^0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad z^0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad u^0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Parameters:

$$\lambda = 0.5, \quad \rho = 1$$

Compute:

1. The updated w^1
2. The updated z^1 using the soft-thresholding operator

3. The updated dual variable u^1

Solution:

ADMM updates:

$$w^{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho(z^k - u^k))$$

First compute

$$A^T A = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}$$

$$A^T b = \begin{bmatrix} 1 \\ 4 \end{bmatrix}$$

Thus

$$A^T A + I = \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix}$$

$$(A^T A + I)^{-1} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{5} \end{bmatrix}$$

Since $z^0 = u^0 = 0$,

$$w^1 = \begin{bmatrix} 1/2 \\ 4/5 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix}$$

Soft threshold:

$$z^1 = S_{\lambda/\rho}(w^1)$$

$$S_{0.5}(x) = \text{sign}(x) \max(|x| - 0.5, 0)$$

Thus

$$z^1 = \begin{bmatrix} 0 \\ 0.3 \end{bmatrix}$$

Dual update:

$$u^1 = u^0 + w^1 - z^1 = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

14. Programming: Empirical Comparison of SGD, Nesterov and L-BFGS

1. Generate a synthetic classification dataset:

$$x \in \mathbb{R}^{50}$$

Class label:

$$y = \text{sign}(w^T x + 0.5 \sin(v^T x))$$

where w, v are random vectors.

Use $N = 6000$ samples and split into 80% training and 20% test.

2. Train the following neural network:

$$50 \rightarrow 128 \rightarrow 64 \rightarrow 2$$

ReLU activations.

3. Train the model using three optimizers:

- SGD ($\text{lr} = 0.01$)
- SGD with Nesterov momentum (momentum = 0.9)
- L-BFGS (history size = 10)

4. During training record:

- Training loss vs epoch
- Gradient norm vs epoch
- Parameter update magnitude vs epoch

5. Plot the loss surface trajectory by projecting parameters onto first two PCA components of parameter history.

6. Compare:

- convergence speed
- stability of updates
- final test accuracy

Provide a brief research-style discussion on how curvature information in L-BFGS and look-ahead gradients in Nesterov influence optimization.

Solution:

Expected observations:

- L-BFGS shows rapid initial decrease in loss due to approximate second-order curvature information.

- Nesterov momentum stabilizes oscillations and converges faster than vanilla SGD.
- SGD exhibits noisier trajectory but may generalize better in some regimes.
- PCA trajectory plots typically show smoother paths for Nesterov and sharper directional moves for L-BFGS.

15. Programming + Conceptual Question: Autoencoders with Regularization

An *autoencoder* is a neural network trained to reconstruct its input. Given an input vector $\mathbf{x} \in \mathbb{R}^d$, the encoder compresses the input into a latent representation $\mathbf{z}$, and the decoder reconstructs the input from this representation:

$$\mathbf{z} = f_{\theta}(\mathbf{x}), \quad \hat{\mathbf{x}} = g_{\phi}(\mathbf{z}).$$

The model is trained by minimizing a reconstruction loss such as Mean Squared Error (MSE):

$$\mathcal{L}_{\text{rec}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\|_2^2.$$

In this problem, you will implement and analyze three versions of an autoencoder:

1. a **naive autoencoder** with no regularization,
2. an autoencoder with **dropout regularization**,
3. a **denoising autoencoder** using noise injection at the input.

You may use any deep learning library such as PyTorch or TensorFlow.

Architecture to Use

Assume the input dimension is d . Use the following fully connected autoencoder architecture:

$$d \rightarrow 128 \rightarrow 64 \rightarrow m \rightarrow 64 \rightarrow 128 \rightarrow d,$$

take m maybe 16, 32, or 64
where:

- the encoder is: $d \rightarrow 128 \rightarrow 64 \rightarrow m$,
- the decoder is: $m \rightarrow 64 \rightarrow 128 \rightarrow d$,
- m is the latent dimension, with $m < d$,
- ReLU activations are used in hidden layers,
- Decide the appropriate activation function for decoder output yourself.

Dataset: Use the standard dataset of MNIST. Flatten each input sample into a vector of size d before feeding it to the network.

(a) **Naive Autoencoder Implementation**

- (i) Implement the naive autoencoder using the architecture given above.
- (ii) Train the model using reconstruction loss.
- (iii) Plot the training loss and validation loss across epochs.
- (iv) Show a few examples of original inputs and their reconstructions on the test set.

(b) **Conceptual Analysis of the Naive Autoencoder**

Answer the following:

- (i) Even with a bottleneck latent layer (with dimension m), why might the model still memorize useful patterns specific to the training set?
- (ii) What signs in the training and validation loss curves would indicate overfitting?

(c) **Autoencoder with Dropout**

Modify the model by adding dropout after each hidden layer in the encoder and decoder. Let the dropout probability be $p = 0.3$ (or another reasonable value).

- (i) Implement this dropout-based autoencoder.
- (ii) Train it on the same dataset under the same experimental settings.
- (iii) Plot the training and validation loss curves.

(d) **Conceptual Questions on Dropout**

Answer the following:

- (i) How does dropout modify the hidden activations during training?
- (ii) Why does dropout reduce overfitting?
- (iii) Why is dropout turned off during inference/testing?

(e) **Denoising Autoencoder via Input Noise Injection (Similar to Question-12) or modify the images using the methods mentioned in Question-2 and then flatten the images**

Instead of feeding the clean input $\mathbf{x}$ directly to the model, create a corrupted input $\tilde{\mathbf{x}}$ and train the model to reconstruct the original clean input $\mathbf{x}$:

$$\tilde{\mathbf{x}} = \mathbf{x} + \epsilon,$$

where ϵ is random noise (for example, Gaussian noise with small variance, or masking noise that sets a fraction of inputs to zero).

- (i) Implement a denoising autoencoder using the same architecture.
- (ii) Train it using noisy inputs and clean targets.
- (iii) Plot the training and validation loss curves.
- (iv) Show sample noisy inputs, clean inputs, and reconstructed outputs.

(f) **Conceptual Questions on Noise Injection**

Answer the following:

- (i) Why does training on noisy inputs help the model learn more robust features?

- (ii) How is noise injection different from dropout in terms of where randomness is introduced?
- (g) **Experimental Comparison** preferably on **wandb**.
Compare the following three models:
- Naive autoencoder
 - Autoencoder with dropout
 - Denoising autoencoder
- In your comparison, include:
- (i) reconstruction loss on training data,
 - (ii) reconstruction loss on validation/test data,
 - (iii) visual quality of reconstructions,
 - (iv) which model generalizes best and why.